

Requirement - Nobisoft

Visual Search Engine



1. Tổng quan

1.1 Mô tả

Hệ thống tìm kiếm hình ảnh thông minh hỗ trợ 3 phương thức tìm kiếm:

Search by Image: Upload ảnh → tìm các ảnh tương tự về nội dung / hình dạng

Search by Text (Semantic): Nhập mô tả bằng ngôn ngữ tự nhiên (ví dụ: "a red car on a street") → tìm ảnh có nội dung tương ứng

Search by Text (OCR): Nhập text xuất hiện trong ảnh (ví dụ: "Nike", "Sale 50%", "STOP") → tìm ảnh chứa dòng chữ đó

1.2 Nền tảng

Web App (React + Vite)

1.3 Quy mô dữ liệu

Hỗ trợ tìm kiếm trên **50,000 — 100,000** ảnh

Dataset gợi ý: [Unsplash Lite](#) (~25K ảnh), [Open Images](#), [DeepFashion](#) hoặc dữ liệu ảnh cá nhân

1.4 Thời gian triển khai

8 tuần (2 tháng)

3 team: AI, Backend, Frontend

2. Yêu cầu chức năng (Functional Requirements)

FR-01: Đăng ký & Đăng nhập (Authentication)

ID	User Story	Priority
FR-01.1	Là người dùng mới, tôi muốn đăng ký tài khoản bằng Email + Password	Must

ID	User Story	Priority
FR-01.2	Là người dùng đã có tài khoản, tôi muốn đăng nhập bằng Email + Password	Must
FR-01.3	Hệ thống sử dụng JWT Access Token để xác thực API	Must

Acceptance Criteria:

Password phải được hash bằng bcrypt

Access Token hết hạn sau 24 giờ

API trả về 401 khi token không hợp lệ hoặc hết hạn

FR-02: Tìm kiếm bằng Hình ảnh (Visual Search)

ID	User Story	Priority
FR-02.1	Là người dùng, tôi muốn upload 1 bức ảnh từ máy tính để tìm các ảnh tương tự	Must
FR-02.2	Là người dùng, tôi muốn kéo thả ảnh (Drag & Drop) vào ô tìm kiếm thay vì chọn file	Must
FR-02.3	Kết quả tìm kiếm phải hiển thị điểm tương đồng (Similarity Score %) cho mỗi ảnh	Must
FR-02.4	Là người dùng, tôi muốn crop (khoanh vùng) một phần của bức ảnh đã upload để chỉ tìm kiếm đối tượng trong vùng đó	Should

Acceptance Criteria:

Hỗ trợ format: JPG, PNG, WebP. Kích thước tối đa: 10MB

Trả về Top-20 kết quả mặc định

Thời gian phản hồi < **3 giây**

Crop tool sử dụng thư viện react-easy-crop

FR-03: Tìm kiếm bằng Text

ID	User Story	Priority
FR-03.1	Là người dùng, tôi muốn nhập mô tả nội dung (ví dụ: "sunset on the beach") để tìm ảnh có nội dung tương ứng	Must

ID	User Story	Priority
FR-03.2	Là người dùng, tôi muốn tìm ảnh chứa dòng chữ cụ thể (ví dụ: "STOP", "Sale") bằng cách nhập text đó	Must
FR-03.3	Hệ thống cho phép chọn chế độ tìm kiếm : "Nội dung ảnh" (Semantic) hoặc "Chữ trong ảnh" (OCR)	Must

Acceptance Criteria:

Semantic search sử dụng CLIP text encoder (cùng vector space với image embedding)

OCR search sử dụng PostgreSQL full-text search (ILIKE hoặc tsvector)

Hỗ trợ tiếng Anh (bắt buộc). Tiếng Việt là bonus

Thời gian phản hồi < 2 giây

FR-04: Hiển thị Kết quả & Tương tác

ID	User Story	Priority
FR-04.1	Kết quả hiển thị dạng lưới ảnh (Masonry Grid)	Must
FR-04.2	Hỗ trợ phân trang hoặc Infinite Scroll để xem thêm kết quả	Must
FR-04.3	Click vào ảnh để xem chi tiết : ảnh phóng to, metadata (kích thước, nguồn), OCR text (nếu có)	Must
FR-04.4	Nút "Tìm ảnh tương tự" từ một ảnh trong kết quả (search-from-result)	Must
FR-04.5	Hiệu ứng Skeleton Loading khi đang tải ảnh	Should
FR-04.6	Lưu ảnh vào Bookmark để xem lại sau	Could

Acceptance Criteria:

Grid layout sử dụng react-masonry-css hoặc CSS Grid

Phân trang mỗi trang 20 ảnh. Hoặc Infinite Scroll nếu kịp tiến độ

Detail view hiển thị dạng Modal

FR-05: Admin Dashboard

ID	User Story	Priority
FR-05.1	Là Admin, tôi muốn xem Dashboard tổng quan : tổng số ảnh đã index, tổng users	Must
FR-05.2	Là Admin, tôi muốn trigger Indexing cho một batch ảnh mới vào hệ thống	Must
FR-05.3	Là Admin, tôi muốn xem trạng thái Indexing (đang chạy / hoàn thành / lỗi)	Must

Acceptance Criteria:

Phân biệt Admin/User bằng trường role trong DB

Indexing status: FE polling GET /admin/indexing/status mỗi 5 giây

Hiển thị danh sách User (chỉ xem, không cần chức năng khoá/xoá)

3. Yêu cầu phi chức năng (Non-Functional Requirements)

NFR-01: Hiệu năng (Performance)

ID	Yêu cầu	Chỉ số
NFR-01.1	Thời gian phản hồi Search API (Image)	< 3 giây
NFR-01.2	Thời gian phản hồi Search API (Text)	< 2 giây
NFR-01.3	Thời gian Indexing 1 ảnh	< 5 giây
NFR-01.4	Trang web load lần đầu (LCP)	< 5 giây

NFR-02: Bảo mật (Security)

ID	Yêu cầu
NFR-02.1	Tất cả API phải kiểm tra Authentication (trừ Login/Register)
NFR-02.2	Validate file upload: chỉ cho phép image MIME type, giới hạn kích thước 10MB
NFR-02.3	Password hash bằng bcrypt

4. Kiến trúc Hệ thống

Đính kèm ảnh sau

5. Phân công theo Team — Sprint Plan (8 tuần)

5.1 Team AI

Tuần	Task	Output
W1	Nghiên cứu CLIP model. Chạy thử trên 10 ảnh mẫu, hiểu input/output	Notebook demo: ảnh → vector, text → vector
W2	Viết module <code>embed_image(image_path) → vector</code> và <code>embed_text(query) → vector</code>	2 Python functions đã test
W3	Tích hợp EasyOCR. Viết module <code>extract_text(image) → list[str]</code>	1 Python function đã test
W4	Viết script Batch Indexing : đọc thư mục ảnh → gọi embed + OCR → insert Qdrant + Postgres	Script chạy được trên 1,000 ảnh
W5- W6	Chạy Batch Indexing toàn bộ dataset (50K-100K ảnh). Debug lỗi, xử lý ảnh hỏng	Vector DB + Postgres đã có dữ liệu
W7- W8	Đo chất lượng search (thử 50 query, đánh giá kết quả). Viết báo cáo kỹ thuật	Báo cáo Precision@10, demo cases

5.2 Team Backend

Tuần	Task	Output
W1	Setup FastAPI project + Docker Compose (FastAPI + Postgres + Qdrant). Thiết kế DB Schema	Repo + docker-compose.yml chạy được
W2	Auth module: Register, Login, JWT middleware	POST /auth/register, POST /auth/login
W3	Search API — Image: nhận file ảnh → gọi AI embed → query Qdrant → trả kết quả	POST /search/image
W4	Search API — Text: semantic search + OCR search (query Postgres)	GET /search/text?q=...&mode=semantic
W5	Indexing API: trigger batch indexing (BackgroundTasks), API xem trạng thái	POST /admin/index, GET /admin/index/status
W6	Admin API: stats, user list. Search History API	GET /admin/stats, GET /history
W7	Viết API Docs (Swagger tự động từ FastAPI). Fix bugs từ FE integration	Swagger UI hoàn chỉnh
W8	Integration testing. Fix edge cases (ảnh lỗi, query rỗng, file quá lớn)	Hệ thống ổn định

5.3 Team Frontend

Tuần	Task	Output
W1	Setup React + Vite. Xây Design System (Color palette, Typography, Button, Input, Card components)	Repo + Style Guide cơ bản
W2	Trang Login / Register. Lưu JWT vào localStorage, tạo Auth Guard	2 trang Auth hoàn chỉnh
W3	Trang Search chính: Search Bar (toggle 3 chế độ), Drag & Drop upload zone	Trang Home với ô search
W4	Search Results: Masonry Grid Layout + Phân trang + Skeleton Loading	Hiển thị kết quả
W5	Image Detail Modal: zoom, metadata, OCR text, nút "Tìm tương tự". Tích hợp Crop tool	Modal chi tiết + Crop tool
W6	Trang Search History + Admin Dashboard (stats cards, indexing status polling)	2 trang phụ
W7	Responsive Mobile. Polish UI (hover effects, transitions, loading states)	UI mượt trên mọi màn hình
W8	Fix bugs. Tối ưu (lazy-load ảnh, code splitting). Chuẩn bị Demo	Sản phẩm sẵn sàng demo

6. Tech Stack

Layer	Công nghệ	Ghi chú
AI	CLIP (ViT-B/32) via HuggingFace transformers	Sử dụng pretrained model
AI	EasyOCR	Hỗ trợ tiếng Việt, dễ cài đặt
BE	Python/Java/.NET	Cùng ngôn ngữ với AI module, tích hợp nhanh
BE	SQLAlchemy + Alembic	ORM + Database migration
DB	PostgreSQL	Lưu users, metadata, OCR text, search history
DB	Qdrant (Docker)	Vector search, REST API đơn giản
Storage	Local Disk (Docker volume)	Lưu ảnh gốc
FE	React 18 + Vite	Build nhanh, DX tốt
FE	Axios + React Query (TanStack Query)	API calls + client-side caching
FE	react-masonry-css	Masonry Grid layout
FE	react-easy-crop	Image cropping tool

Layer	Công nghệ	Ghi chú
DevOps	Docker Compose	3 containers: app + postgres + qdrant

7. Tiêu chí Nghiệm thu (Definition of Done)

Milestone 1 — Tuần 4 (Mid-term Review)

Hệ thống Auth hoạt động (Register / Login / JWT)

Search bằng Image trả về kết quả đúng trên $\geq 1,000$ ảnh

Search bằng Text (semantic) trả về kết quả có ý nghĩa

FE hiển thị kết quả dạng Grid

Toàn bộ hệ thống chạy được trên Docker Compose (docker-compose up)

Milestone 2 — Tuần 8 (Final Demo)

3 chế độ Search hoạt động đầy đủ (Image / Semantic Text / OCR Text)

Dataset $\geq 50,000$ ảnh đã được index

Thời gian search < 3 giây

Drag & Drop upload hoạt động

Masonry Grid Layout + Phân trang + Skeleton Loading

Crop tool hoạt động

Admin Dashboard hiển thị stats + indexing status

Responsive trên Mobile

API Docs (Swagger) + README setup

8. Rủi ro & Giải pháp

#	Rủi ro	Giải pháp
R1	CLIP model chạy chậm trên máy không có GPU	Dùng model nhỏ ViT-B/32 thay vì ViT-L/14. Hoặc dùng Google Colab free GPU cho giai đoạn nghiên cứu
R2	Integration giữa 3 team bị delay vì API chưa sẵn sàng	BE phải có Mock API + Swagger Docs từ tuần 2. FE dùng mock data để phát triển song song

#	Rủi ro	Giải pháp
R3	Batch indexing 50K+ ảnh tốn nhiều giờ	Chạy qua đêm. Chia thành nhiều batch nhỏ (5K ảnh/batch). Có script retry khi lỗi
R4	Intern chưa quen Docker	Dành buổi đầu tiên hướng dẫn Docker 101. Viết sẵn script make dev để chạy 1 lệnh